Internet Applications

Introduction to Express.js

Suhel Hammoud

What Will We Learn?

- Follow an HTTP request from a client to a server and back.
- Understand what Express adds to Node.js.
- Write a Hello World server in TypeScript.
- Build and call a `/health` endpoint.

Start with One HTTP Request

Imagine a client asks whether our application is running:

```
GET /health HTTP/1.1
Host: localhost:3000
```

- `GET` is the method: we want to retrieve information.
- `/health` is the path: what we are asking for.
- `localhost` means this computer.
- `3000` is the port where our server will listen.

The Server Sends a Response

A simplified response from the app we will build:

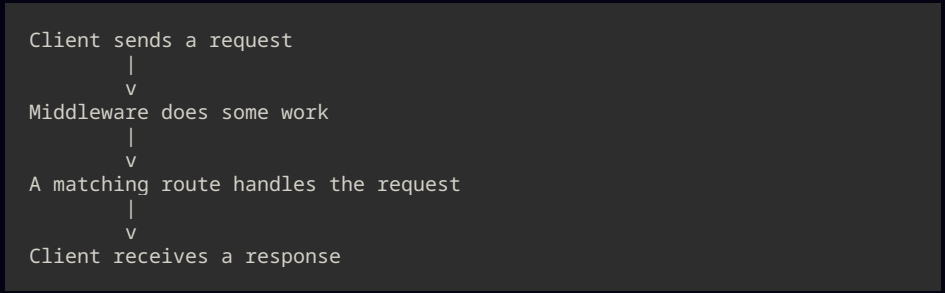
```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status":"ok"}
```

- `200 OK`: the request succeeded.
- `Content-Type`: the body contains JSON.
- The blank line separates the headers from the body.
- `{"status":"ok"}` is the response body.

What Is Express.js?

- Express is a web framework that runs on Node.js.
- It helps us decide how to handle HTTP requests.
- We connect a method and a path to a JavaScript function.
- That function can send text, JSON, or other responses.
- In this lesson, we will use Express 5 with TypeScript.

Follow the Request Through Express



We will follow this path using `GET /health`.

Create a Practice Project

Use a new folder with Node.js and npm already installed.

```
mkdir express-introduction
cd express-introduction
npm init -y
npm pkg set type=module
mkdir src
```

- Use a currently supported Node.js LTS release.
- Express 5 requires Node.js 18 or newer.
- `src` will contain our TypeScript code.

Install Express and the TypeScript Tools

Run these commands in the practice folder:

```
npm install express@5
npm install --save-dev typescript tsx \
  @types/node @types/express@5
```

- `express@5`: the web framework for our server.
- `typescript`: checks our types.
- `tsx`: runs our TypeScript code.
- The `@types` packages describe Node.js and Express for the checker.

Copy the TypeScript Configuration

Create `tsconfig.json` in the practice folder:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "types": ["node"]
  },
  "include": ["src/**/*.ts"]
}
```

These are the same starter settings as our TypeScript lesson.

Hello World in TypeScript

Create `src/index.ts`:

```ts
import express from "express";

const app = express();

app.get("/", (req, res) => {
  res.send("Hello World!");
});

app.listen(3000, () => {
  console.log("Server: http://localhost:3000");
});
```

Read the First Two Lines

```
import express from "express";

const app = express();
```

- `import` loads Express from the package we installed.
- `express()` creates our application.
- We store that application in a variable called `app`.

Connect a Request to a Function

```
app.get("/", (req, res) => {
  res.send("Hello World!");
});
```

- `app.get` handles a `GET` request.
- `"/"` is the root path of our application.
- `(req, res) => { ... }` is the function that handles it.
- This connection between a request and a handler is a **route**.

Meet req and res

```
app.get("/", (req, res) => {
  res.send("Hello World!");
});
```

- `req` is the **request**: information sent by the client.
- `res` is the **response**: tools for replying to the client.
- `res.send()` sends our greeting and ends the response.

Express's types let TypeScript infer the types of `req` and `res`.

Start Listening for Requests

```
app.listen(3000, () => {
  console.log("Server: http://localhost:3000");
});
```

- `listen(3000)` starts the server on port `3000`.
- The function runs when the server starts listening successfully.
- The message in the terminal tells us where to call the server.
- Register our routes before calling `listen()` in this example.

Check and Run the Server

In terminal 1, from the practice folder:

```
npx tsc --noEmit
npx tsx src/index.ts
```

Expected startup message:

```
Server: http://localhost:3000
```

- Keep this terminal open while using the server.
- The process stays running so it can answer requests.
- Press Ctrl+C when you want to stop it.

Call Hello World

Open a browser at `http://localhost:3000/`.

Or, in terminal 2, use `curl`:

```
curl -i http://localhost:3000/
```

- `curl` is a command-line HTTP client.
- It sends a `GET` request by default.
- `-i` shows the response headers as well as the body.
- Look for `200 OK` and the body `Hello World!`.

What Is a Health Endpoint?

- An **endpoint** is a method and path clients can call.
- Our new endpoint will be `GET /health`.
- It will reply with a small message: `{"status":"ok"}`.
- This tells us that the application can answer this request.
- This simple check does not test a database or other services.

Add the Health Route

Add this below the Hello World route, before `app.listen()`:

```
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

- This route matches `GET /health`.
- `res.json()` sends an object as JSON.
- Express sets the JSON content type for us.
- The response uses status `200` by default.

A JavaScript Object Becomes JSON

In our TypeScript code:

```
{ status: "ok" }
```

In the HTTP response body:

```
{"status":"ok"}
```

- JSON is a text format used to exchange data.
- JSON property names use double quotes.
- `res.json()` handles this conversion for us.

Restart and Call the Health Endpoint

In terminal 1, stop the server with `Ctrl+C`, then run:

```
npx tsc --noEmit
npx tsx src/index.ts
```

In terminal 2:

```
curl -i http://localhost:3000/health
```

Look for `200 OK`, a JSON content type, and `{"status":"ok"}`.

Restart after code changes when using this run command.

What Is Middleware?

- Middleware is a function that can work on a request.
- For example, it can print which request arrived.
- It can send a response or pass the request onward.
- Calling `next()` passes control to the next matching handler.

We will add one middleware before our routes.

Print Each Incoming Request

Add this immediately after `const app = express();`:

```
app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});
```

- `app.use()` registers middleware; here it runs for every request.
- `req.method` is the HTTP method, such as `GET`.
- `req.path` is the path, such as `/health`.
- `next()` lets Express continue to the route.

Keep `next()`: without it or a response, the request would wait.

Trace One Request from Start to Finish

After restarting, call `GET /health` again:

1. curl sends GET /health.
2. Express receives the request.
3. Our middleware prints GET /health in terminal 1.
4. next() passes the request onward.
5. The /health route calls res.json().
6. curl shows the response in terminal 2.

The middleware runs first because we registered it before the routes.

The Complete App

Your `src/index.ts` should now look like this:

```
import express from "express";

const app = express();

app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});

app.get("/", (req, res) => { res.send("Hello World!");});

app.get("/health", (req, res) => { res.json({ status: "ok" }); });

app.listen(3000, () => console.log("Server: http://localhost:3000"););
```

If Something Does Not Work

- **Connection refused:** check that the server is running.
- **Port already in use:** stop an earlier server on port `3000`.
- **404 Not Found:** check the path; `/heath` is not `/health`.
- **Request keeps waiting:** check for a missing `next()` or response.
- **Old response appears:** save the file and restart the server.

The server terminal and `curl -i` show different sides of the same request.

References

Hello World (https://expressjs.com/en/starter/hello-world/)

Installing (https://expressjs.com/en/starter/installing/)

Middleware (https://expressjs.com/en/guide/using-middleware/)

Express 5 API (https://expressjs.com/en/5x/api/)